

Table 1: Aggregate compression results excluding 7z.

Dataset	Technique	Original (bytes)	Compressed (bytes)	Ratio	Time (ms)
Random	OpenZL	22,369,280	22,369,536	1.000011	276.915
Random	Gzip	22,369,280	22,376,165	1.000308	630.040
Random	Zstandard	22,369,280	22,369,868	1.000026	19.565
Random	ZIP	22,369,280	22,374,766	1.000245	1185.037
Trend	OpenZL	22,369,280	1,454,615	0.065027	256.617
Trend	Gzip	22,369,280	5,114,980	0.228661	946.561
Trend	Zstandard	22,369,280	3,381,713	0.151177	91.088
Trend	ZIP	22,369,280	5,116,907	0.228747	1263.900

1 Compression Choice for a Streaming Platform

The compression method for a streaming platform should be selected according to the data type and the latency target rather than by compressed size alone. The random-data results show that none of the tested methods can meaningfully reduce unstructured data: each output is approximately the same size as the input. For this type of payload, compression adds processing and framing overhead, so the platform should either disable compression for small messages or use a fast codec only when bandwidth is more important than CPU time.

For structured sensor or telemetry data, the trend-data results are very different. OpenZL produced the smallest aggregate output, approximately 6.5% of the original size, while Zstandard produced approximately 15.1%. Zstandard was substantially faster in the tests, with an aggregate time of 91.088 ms compared with 256.617 ms for OpenZL. This makes Zstandard the best general-purpose choice for a streaming service where predictable latency, moderate CPU usage, and broad deployment support are important. It also has good support across common programming languages and message brokers.

OpenZL is preferable when the stream has a stable, structured layout and the main objective is reducing storage or network volume. Its stronger reduction on the trend data indicates that it can exploit relationships between values, but the additional processing cost should be measured against the service’s latency budget. Compression should normally be performed in batches or records large enough to amortize codec overhead. The recommended policy is therefore to use Zstandard as the default codec, select OpenZL for validated structured streams with high compression value, and avoid compression for already-compressed or effectively random payloads.

2 Kafka versus RabbitMQ for a Priority-Based System

Kafka and RabbitMQ solve related but different messaging problems. Kafka is a distributed append-only event log designed for high-throughput streams, durable retention, and replay. RabbitMQ is a broker built around exchanges, queues, routing rules, acknowledgements, and consumer delivery. The choice for a priority-based system should therefore be based on whether priority is a property of a retained event stream or a property of the next message that a worker must receive.

Kafka can support priority, but priority is not its fundamental delivery semantic. A common design is to publish urgent and normal events to separate topics or partitions and give urgent consumers more capacity. Another option is to maintain separate consumer groups or to use an application-level scheduler. These approaches scale well and preserve Kafka’s retention and replay model, but they require extra design work. A single Kafka partition also provides ordering, so strict priority can conflict with throughput when a lower-priority record appears ahead of an urgent record in the same partition.

RabbitMQ is generally the more direct choice when the system must deliver the highest-priority available job first. Priority queues, acknowledgements, dead-letter exchanges, retry policies, and routing keys provide the controls needed for operational workflows. For example, an emergency alert can be routed to a high-priority queue while routine telemetry is handled by a normal queue. Consumers can acknowledge work only after processing, allowing failed messages to be retried without losing them. The trade-off is that RabbitMQ

Table 2: Kafka and RabbitMQ characteristics for priority-based workloads.

Criterion	Kafka	RabbitMQ
Primary model	Durable, partitioned event log	Routed work queues
Priority	Usually implemented through topics, partitions, or separate consumers	Native queue priorities and routing patterns
Ordering	Strong ordering within each partition	Ordering within a queue, subject to competing consumers and priority
Replay	Built in through retained offsets	Normally requires explicit requeueing or a separate durable design
Throughput	Very high for sequential stream processing	Strong, but optimized for brokered delivery and routing
Best fit	Analytics, event sourcing, telemetry, and fan-out	Commands, jobs, alerts, and urgent work items

does not provide Kafka’s equivalent of a long-lived, easily replayable event history, and very large retained streams are usually better handled by Kafka.

For a priority-based service, RabbitMQ should be selected when the key requirement is immediate, priority-aware task delivery with per-message acknowledgement and retry handling. Kafka should be selected when the system must retain a large stream, support multiple independent consumers, replay historical events, or feed analytics and storage pipelines. A hybrid design is also appropriate: Kafka can retain the authoritative telemetry or event stream, while RabbitMQ can carry derived high-priority commands and alerts to workers. This separates durable event history from time-sensitive operational delivery instead of forcing one broker to provide both semantics.